Design Analysis and Algorithm – Lab Work

Week 6

Question 1:Write a Program to perform Quick Select.

Code:

```c
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            swap(&arr[i], &arr[j]);
            i++;
        }
    }
    swap(&arr[i], &arr[high]);
    return i;
}

// Quick Select (k is 1-based)
int quickSelect(int arr[], int low, int high, int k) {
    if (low <= high) {
        int pivotIndex = partition(arr, low, high);

        if (pivotIndex == k - 1)
            return arr[pivotIndex];
        else if (pivotIndex > k - 1)
            return quickSelect(arr, low, pivotIndex - 1, k);
        else
            return quickSelect(arr, pivotIndex + 1, high, k);
    }
    return -1;
}

int main() {
```

```
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = sizeof(arr) / sizeof(arr[0]);

    int k = 3;

    int result = quickSelect(arr, 0, n - 1, k);

    printf("%d-th smallest element is: %d\n", k, result);
    return 0;
}
```

Output:

```
\Week 6> gcc quickselect.c
PS E:\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\4th Semester\Design Analysis And Algorithm
\Week 6> ./a
3-th smallest element is: 112
PS E:\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\4th Semester\Design Analysis And Algorithm
\Week 6>
```

Time Complexity

The time complexity of the Quick Select algorithm depends on how the pivot divides the array. In the best and average cases, the pivot divides the array into roughly equal halves, reducing the problem size significantly at each step. This leads to a time complexity of O(n) because only one side of the partition is processed recursively instead of both sides as in QuickSort. However, in the worst case, when the pivot is always the smallest or largest element (for example, in a sorted array with poor pivot choice), the algorithm reduces the problem size by only one element each time. This results in a time complexity of $O(n^2)$. Despite this worst-case scenario, Quick Select is efficient in practice due to its linear average performance.

Space Complexity

The space complexity of Quick Select is primarily determined by the recursion stack. In the best and average cases, the recursion depth is logarithmic because the array is divided into smaller portions efficiently, resulting in a space complexity of O(log n). However, in the worst case, where the partitioning is highly unbalanced, the recursion depth can grow linearly, leading to a space complexity of O(n). Apart from the recursion stack, Quick Select is an in-place algorithm and does not require any additional memory, making it space efficient.